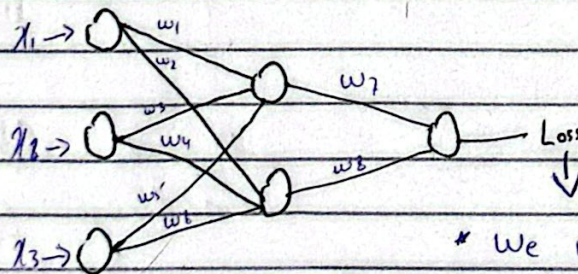


Optimizers

- (1) Gradient Descent (2) Stochastic Gradient Descent
(3) Mini Batch SGD (4) SGD with momentum
(5) Adagrad & RMSProp (6) Adam optimizers.

→ 1: Gradient Descent optimizer:



* We need to reduce this loss in backward propagation

* Optimizer is responsible for reducing this loss function in BP.

* In BP main AIM of the optimizer is to update the weights.

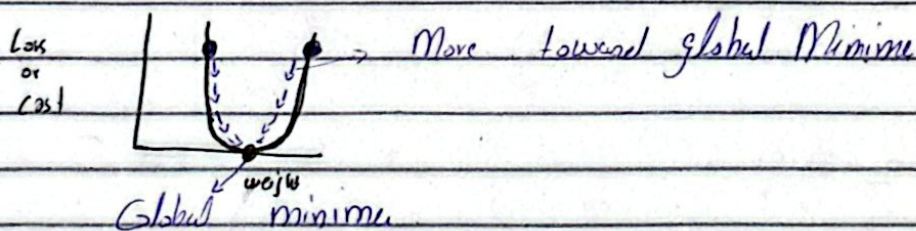
* In gradient Descent we take all the datapoints in one go.

Weight Update formula:

$$\Rightarrow W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial \text{Loss}}{\partial W_{\text{old}}}$$

* This formula is going to change for every optimizer.

→ Gradient Descent:



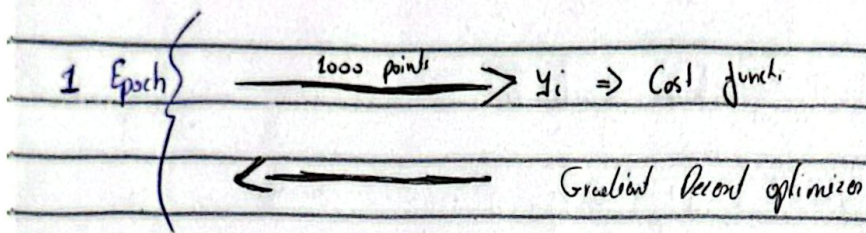
⇒ Let Take MSE:

$$\text{Loss fn} = (y - \hat{y})^2, \quad \text{cost fn} = \frac{1}{n} \sum_{i=1}^n (y - \hat{y})^2$$

⇒ Epoch:

* One forward propagation with all the data points (cost fn) & one backward propagation is an Epoch.

→ I have 1000 Data points, I do forward propagation of all available cost fn & use optimizer to reduce cost fn in backward propagation & weights will get updated.



* This process of One forward propagation with all datapoints & one backward propagation is one epoch.

* & we will continue this (any number of epochs) till we get optimized weights.

⇒ Iteration:

~~1 Epoch = 1 Iteration~~

* Epoch $\Rightarrow$ Whole dataset (e.g. all the 2000 points)

* One complete pass through entire Training dataset.

* Iteration: One Training step in which model process one batch of training data, calculate loss, compute gradient & update weights.

e.g. Dataset: 1000 samples/datapoints.

1 Epoch = Model see all 1000 samples

1 iteration = Model process one batch & update

⇒ Dataset = 1000 samples
⇒ Batch size = 100

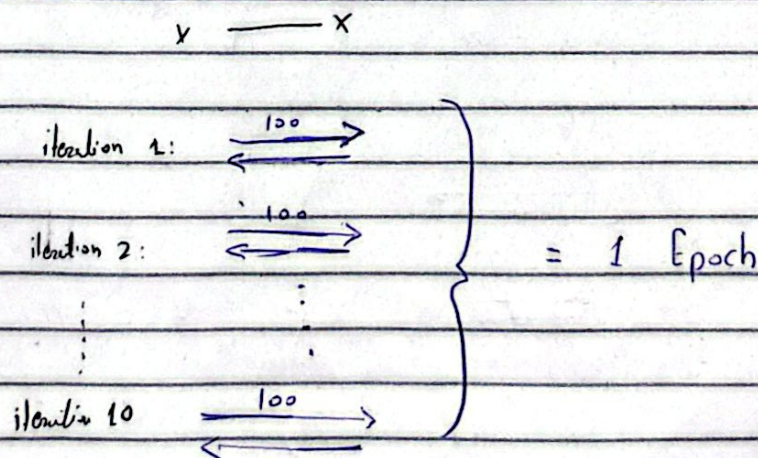
So Batch 1 = 100 samples
Batch 2 = 100 samples
Batch 3 = 100 samples
⋮
Batch 10 = 100 samples

Thus Each batch passes, compute Loss, gradient, & update weights.

★ ★ 1 batch = 1 iteration

★ ★ ★ When model has gone through all 10 batches it has seen all the samples that is 1 epoch.

1 Epoch = 10 iterations.



* In gradient Descent There is One iteration only that takes all The datapoints.

Advantages:

Convergence will happen.

Disadvantages

* Require Huge RAM & GPU

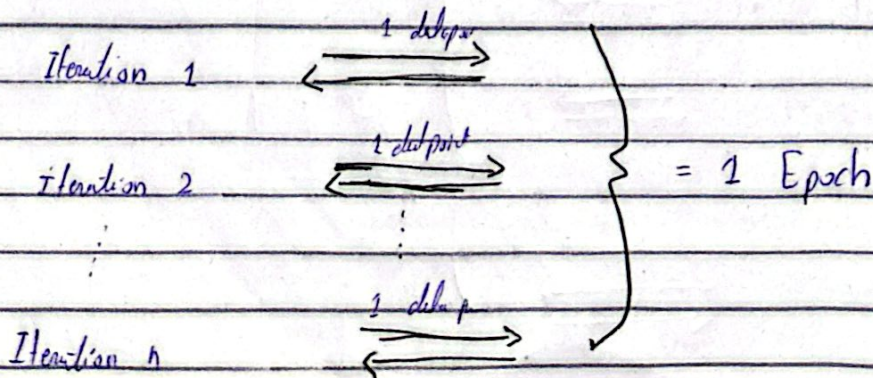
↓
if I have millions data points, then it will take all in forward & then in back compute update weights for all $\rightarrow$ so require resources.

$\hookrightarrow$ 2: Stochastic Gradient Descent (SGD)

Optimizer:

* Here we give Only one data point in One iteration

* for each data point "FP" occurs & calculate Loss & in "BP" using "SGD" update weights.



→ Since we have 1000 data points
so 1 Epoch = 1000 iterations

Advantages:

(1) Solve The resource issue (As one record/data points at a time).

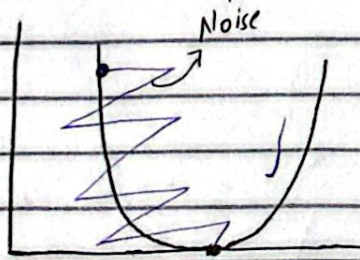
Disadvantage:

(1) Takes Huge amount of time.

(2) Convergence will also Take more time

(3) Noise get Introduced.

Path of convergence is not very smooth



• Observe The convergence is not smooth & will take time

x — x

↳ 3: Mini Batch (SGD)

* Here we use Batch
if I have dataset of 100000 samples
& Batch-size = 1000

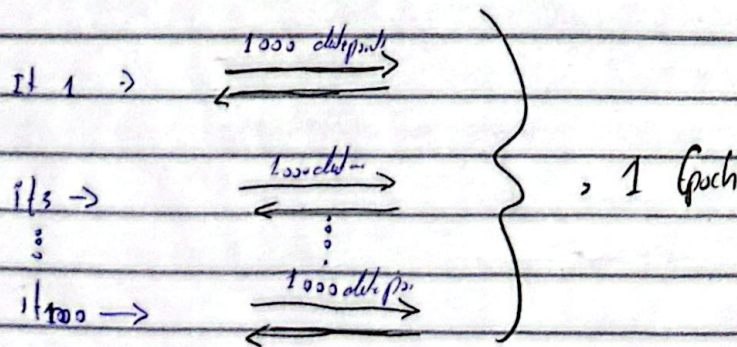
So Now each iteration will have 1000 samples.

$$\text{nmbr of iterations} = \frac{\text{datapoints}}{\text{Batch size}}$$

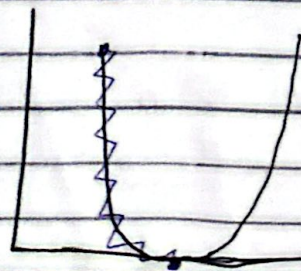
$$= \frac{10000}{100} = 100 \text{ iteration}$$

* So 1 Epoch will have 100 iteration.

e.g after 100 iteration all The data have been seen by model.



* This will Reduce the Noise



* See The Noise got reduced.

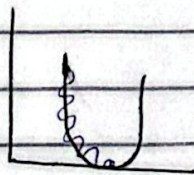
Advantages:

- (1) Compared SGD convergence speed will increase
- (2) Noise will be reduced
- (3) Efficient Resource Usage (RAM)

Disadvantage

- (1) Noise still exists.

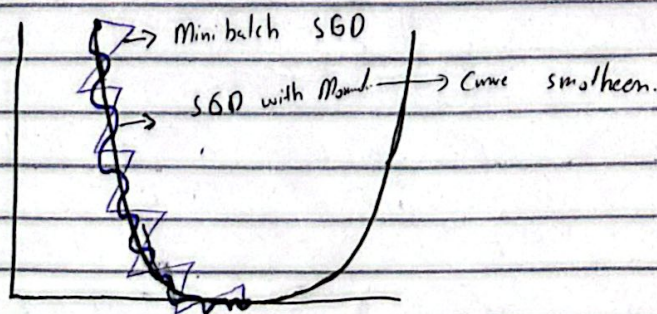
⇒ Our Next Aim is to smooth the Noise.



For This we use SGD with Momentum.

x ——— x

⇒ 4: SGD with Momentum:



Weight Update Formula:

$$W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial \text{Loss}}{\partial W_{\text{old}}}$$

Bias Update Formula:

$$b_{\text{new}} = b_{\text{old}} - \eta \frac{\partial \text{Loss}}{\partial b_{\text{old}}}$$

Let's use Another Representation for This

$$\Rightarrow W_t = W_{\text{new}}$$

$$\Rightarrow W_{t-1} = W_{\text{old}} \quad (\text{Previous weight})$$

$$* W_t = W_{t-1} - \eta \frac{\partial \text{Loss}}{\partial W_{t-1}}$$

$\Rightarrow W_t$ depends upon W_{t-1}

$\Rightarrow$ New weight depends upon previous weight.

$\Rightarrow$ we use Exponential Weighted Average for smoothing.

⇒ Exponential Weight Average:

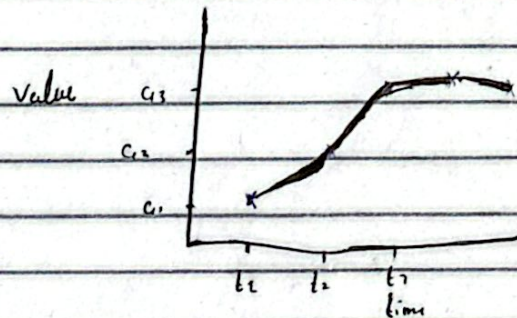
Time = t_1 t_2 t_3 t_n

Values = c_1 c_2 c_3 c_n

Value at time t_i is " c_i "

So

$$\Rightarrow V_{t_1} = c_1$$



⇒ To apply Smoothing we will apply some parameter.

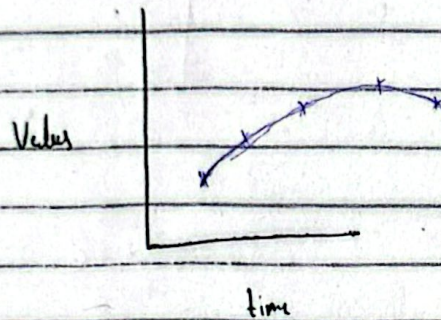
⇒ for Value at Time t_2 , we will apply parameter " β " & This " β " controls The Smoothing function

So ⇒

$$V_{t_2} = \beta * V_{t_1} + (1 - \beta) * c_2$$

∴ β ranges b/w 0-1.

* After Applying This Our graph become smooth



$$\Rightarrow V_{t2} = \beta * V_{t1} + (1 - \beta) * a_2$$

suppose $\beta = 0.95$

$$\Rightarrow V_{t2} = (0.95) * V_{t1} + (0.05) * a_2$$

↓

Previous point

↓

Current point

* Observe when we Make " β " large i.e. 0.95 The previous point get more weight & current point get less weight

Mean at large " β " previous point controls the smoothing.

$$\Rightarrow V_{t3} = \beta * V_{t2} + (1 - \beta) * a_3$$

$$\text{as } V_{t2} = (0.95) * V_{t1} + (0.05) * a_2$$

$$S_0 = V_{13} \times 0.95 \times [0.95 \times V_{11} + (0.05) \times 0.1] + (0.05)$$

Advantages

- (1) Reduces Noise
- (2) Quick Convergence.

x — x

⇒ 5: Adagrad (Adaptive Gradient Descent):

$$W_t = W_{t-1} - \eta \frac{\partial \text{Loss}}{\partial W_{t-1}}$$

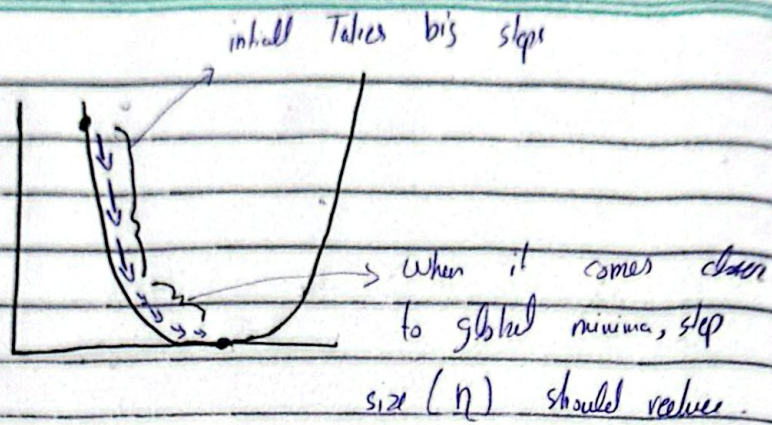
∴ η = Learning Rate

⇒ η is fixed, when convergence happens

⇒ What if we make it Dynamic.

*** Initially when point is very far from global minima, it should take bigger steps, & as come closer to global minima the steps becomes smaller.

*** As Convergence happens Learning Rate should change.



$\Rightarrow$ So

$$w_t = w_{t-1} - \eta' \frac{\partial \text{Loss}}{\partial w_{t-1}}$$

$\therefore \eta' \Rightarrow$ Dynamic Learning Rate

$$\eta' = \frac{\eta}{\sqrt{\alpha_t + \epsilon}}$$

$\therefore \epsilon = \text{Epsilon}$

$$\alpha_t = \sum_{i=1}^t \left(\frac{\partial \text{Loss}}{\partial w_i} \right)^2$$

as $\eta' \downarrow \downarrow \downarrow$, $\alpha_t \uparrow \uparrow$

Suppose

$$\begin{array}{ccc} \text{at } t=1 & , & \text{at } t=2 & , & \text{at } t=3 \\ \eta = 0.01 & & \eta = 0.005 & & \eta = 0.003 \end{array}$$

⇒ observe Learning Rate keeps on decreasing.

Disadvantage:

* In Deep neural work for η' there is a possibility that it become very small value, approximately equal to zero.

→ suppose $\eta' \approx 0$
then
$$w_t = w_{t-1} - \eta' \frac{\partial \text{Loss}}{\partial w_{t-1}}$$

$$\Rightarrow w_t = w_{t-1} - (0) \frac{\partial \text{Loss}}{\partial w_{t-1}}$$

$$\Rightarrow w_t = w_{t-1}$$

↪ again the same problem
weight updation will not happen.

↳ 6: AdaDelta & RMSProp:

* Here we will fix the problem of η' become very small, approach to zero.

* Here instead of ' α_t ' & we will use ' S_{dw} '.

So:

$$\eta' = \frac{\eta}{\sqrt{S_{dw} + \epsilon}}$$

$\therefore S_{dw} \Rightarrow$ Exponential Weighted Average

* We are going to smooth update of " α_t "

[at $t_1 \Rightarrow S_{dw,t} = 0$]

$$S_{dw,t} = \beta * S_{dw,t-1} + (1-\beta) \left(\frac{\partial \text{Loss}}{\partial w_{t-1}} \right)^2$$

* using the same formula

* here we are restricting the chain Rule Value to be small with $(1-\beta)$

* It is Dynamic Learning Rate
* Also smooth with Exponential Weighted Average

$$x \longrightarrow x$$

↳ 7: Adam Optimizer:

* Best optimizer

↳ It is best aspects of all previous optimizer.

1st $\Rightarrow$ We got less Noise & smooth curve in "SGD with Momentum"

2nd $\Rightarrow$ Dynamic Learning Rate with smooth that we had in "RMSProp"

So adam is combination of Both.

$$w_t = w_{t-1} - \eta' \text{V}_{dw}$$

$$b_t = b_{t-1} - \eta' \text{V}_{db}$$

- * It is Dynamic Learning Rule
- * Also smooth with Exponentiated Weighted Avg.

x — x

→ 7: Adam Optimizer:

- * Best optimizer

→ It is best aspects of all previous optimizers.

1st ⇒ We got Less Noise & smooth curve
as in "SGD with Momentum"

2nd ⇒ Dynamic Learning Rule with smoothing
that we had in "RMSProp"

So adam is combination of Both.

$$* \quad w_t = w_{t-1} - \eta' V_{dw}$$

$$* \quad b_t = b_{t-1} - \eta' V_{db}$$

$$\Rightarrow w_t = w_{t-1} - \eta' \nabla_{w_t}$$

$$\therefore \eta' = \frac{\eta}{\sqrt{Sdw_t + \epsilon}}$$

at 1st Time stamp $\Rightarrow Sdw_t = 0$

$$\text{at 2nd Time stamp} \Rightarrow Sdw_t = \beta * Sdw_{t-1} + (1-\beta) \left(\frac{\partial L}{\partial w_{t-1}} \right)$$

$$\therefore \nabla_{w_t} = \beta * \nabla_{w_{t-1}} + (1-\beta) \frac{\partial \text{Loss}}{\partial w_{t-1}}$$

$$\therefore \nabla_{w_t} = \beta * \nabla_{w_{t-1}} + (1-\beta) \frac{\partial \text{Loss}}{\partial w_{t-1}}$$

weight & bias updation with smoothing.

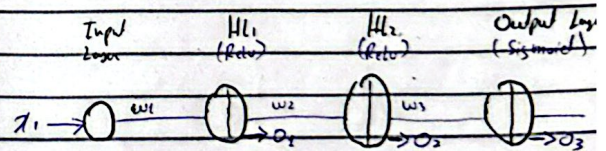
★ Best Optimizer

★ Work for Most/all The Use Cases.

Exploding Gradient Problem.

* Weight updation is happening by very big value, we may never reach the global minimum.

* If we initialized the weight with very very large value then a problem will occur.



I want to update w_2

$$\Rightarrow w_{2, \text{new}} = w_{2, \text{old}} - \eta \left| \frac{\partial \text{Loss}}{\partial w_{2, \text{old}}} \right|$$

Expand chain Rule

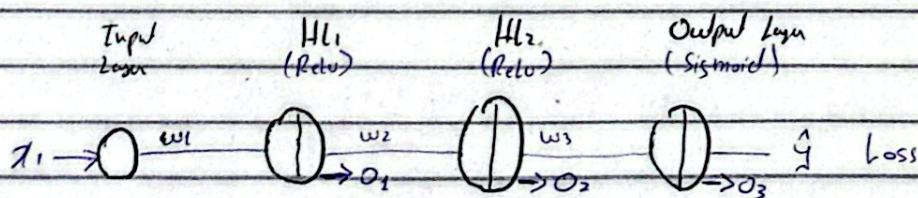
$$\frac{\partial \text{Loss}}{\partial w_{2, \text{old}}} = \frac{\partial \text{Loss}}{\partial o_3} * \left| \frac{\partial o_3}{\partial o_2} \right| * \frac{\partial o_2}{\partial w_{2, \text{old}}} *$$

↳ lets compute this

⇒ Exploding Gradient Problem:

* Weight updation is happening by very big value, we may never reach the global minima

* If we initialized the weight with very very large value then this problem will occur.



I want to update w_1

$$\Rightarrow w_{1_{\text{new}}} = w_{1_{\text{old}}} - \eta \left[\frac{\partial \text{loss}}{\partial w_{1_{\text{old}}}} \right]$$

Expanded chain Rule

$$\frac{\partial \text{loss}}{\partial w_{1_{\text{old}}}} = \frac{\partial \text{loss}}{\partial o_3} * \left[\frac{\partial o_3}{\partial o_2} \right] * \frac{\partial o_2}{\partial o_1} * \frac{\partial o_1}{\partial w_1}$$

↳ let's compute This

$$\frac{\partial \mathcal{L}_3}{\partial \mathcal{O}_2} = \frac{\partial (\mathcal{L}_3)}{\partial (\mathcal{Z})} \times \frac{\partial \mathcal{Z}}{\partial \mathcal{O}_2}$$

$$\frac{\partial \mathcal{L}_3}{\partial \mathcal{O}_2} = [0 - 0.25] \times \frac{\partial (\mathcal{O}_2 \times W_3 + b_3)}{\partial \mathcal{O}_2}$$

$$= [0 - 0.25] \times W_3$$



⇒ As invariably we have initialized weight by very big number

→ So

$$\frac{\partial \mathcal{L}_3}{\partial \mathcal{O}_2} = \text{high (big value)}$$

$$\text{So } \frac{\partial \mathcal{L}_{\text{loss}}}{\partial w_{321}} = \frac{\partial \mathcal{L}_{\text{loss}}}{\partial \mathcal{O}_2} \times \frac{\partial \mathcal{O}_2}{\partial \mathcal{O}_2} \times \frac{\partial \mathcal{O}_2}{\partial \mathcal{O}_2} \times \frac{\partial \mathcal{O}_2}{\partial w_{321}}$$

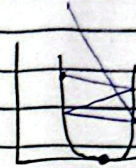
↓ ↓ ↓ ↓
 Very big Value < high high big val big val

$$\text{So } w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial \mathcal{L}_{\text{loss}}}{\partial w_{321}}$$

↓
 Very big Value

$$\text{So } w_{\text{new}} \gg \gg \gg \gg w_{\text{old}}$$

So we will never reach global Min



⇒ So we will use some weight initialization techniques.

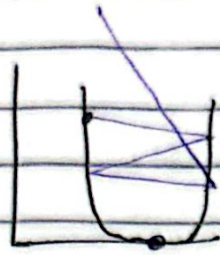
X ——— X

Weight Initialization Techniques:

- 1) Uniform Distribution
- 2) Xavier/Colab Initialization
- 3) Kaiming He Initialization

⇒ There are some of initialization Techniques

So we will never reach global Min



$\Rightarrow$ So we will use some weight initialization techniques.

x ——— x

Weight Initialization Techniques:

- 1) Uniform Distribution
- (2) Xavier / Glorot Initialization
- (3) Kaiming He Initialization

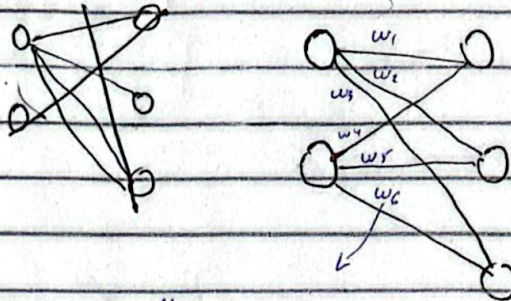
$\Rightarrow$ There are some of weight initialization Techniques.

⇒ Key points :

⇒ Weights should be small

⇒ Weights should not be same ($w_1, w_2, w_3, \dots, w_n$ shouldn't be same initial, zero)

⇒ Weight should have good variance ($w_1, w_2, w_3, \dots, w_n$ should have spread/variance in values)



⇒ All weights shouldn't be same & have variance

⇒ 1: Uniform Distribution:

⇒ We initialize the weight based on of some uniform distribution

⇒ In this uniform distribution we give a range of (a, b)

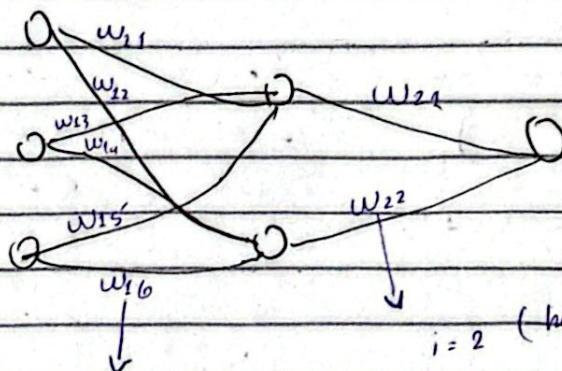
e.g. ⇒ $[-1, 1]$ in uniform dist

Any value in this range has some probability of being selected.

$\Rightarrow W_{ij}$:

$i \Rightarrow$ input layer

$j \Rightarrow$ hidden layer (Number of weight)



$i = 2$ (here the input layer is second one)

$i = 1$

j : number of n

j : number of Neuron

* Above we have : input neuron = 3
output neuron = 1

* $W_{ij} \approx$ uniform Distribution $\left[\begin{matrix} a & , & b \\ \frac{-1}{\sqrt{\text{input}}} & , & \frac{1}{\sqrt{\text{input}}} \end{matrix} \right]$

for above network input = 3

so

$$\left[\frac{-1}{\sqrt{3}} , \frac{1}{\sqrt{3}} \right]$$

$\Rightarrow W_{ij} \approx (a, b)$

a = lower bound

b = upper bound.

* In uniform Dist $\Rightarrow$ There is even dist
min to max $[a-b] \Rightarrow \text{eg} [-1, 1]$

2: Xavier/Glorot Initialization:

$\Rightarrow$ Two Types

a) Xavier Normal

* Weight is initialized from Normal distribution

$$\Rightarrow W_{ij} \approx N(0, \sigma)$$

$\Rightarrow$ in Normal Dist mean = 0
& std = 1

$$\Rightarrow \therefore \sigma = \sqrt{\frac{2}{\text{input} + \text{output}}}$$

(b) Xavier Uniform

* Weight initialized from uniform dist of given range.

$$\Rightarrow W_{ij} \approx U\left(-\frac{b}{\sqrt{6}}, \frac{b}{\sqrt{6}}\right)$$

$$W_{ij} \approx U(a, b)$$

3: Kaiming He Initialization:

a) He Normal

$$\Rightarrow W_{ij} \approx N(0, \sigma)$$

$$\sigma = \sqrt{\frac{2}{\text{input}}}$$

(b) He Uniform

$$\Rightarrow W_{ij} \approx U\left(-\frac{b}{\sqrt{3}}, \frac{b}{\sqrt{3}}\right)$$

$\Rightarrow$ Drop Out

Layer:

$\Rightarrow$ Overfitting: Training Acc $\Rightarrow$ 90%
Test Acc $\Rightarrow$ 60%

$\Rightarrow$ Model is Trained well on Training data
 $\Rightarrow$ Our Aim is to create a general model that perform well on both data

$\Rightarrow$ in ML $\Rightarrow$ Decision Tree $\Rightarrow$ When we decision Tree fully (To last m)
it leads to overfitting $\Rightarrow$ So
perform post pruning & pre pruning
reduce overfitting.

Note: See D
Notes &

$\Rightarrow$ Random Forest: Many Decision
bt we do Feature sampling & Row
means give each Decision Tree diff
sample of Data.

⇒ Drop Out

Layer:

⇒ Overfitting: Training Acc $\Rightarrow$ 90%
Test Acc $\Rightarrow$ 60%

→ Model is Trained well on Training data

→ Our Aim is to create a generalized model that perform well on both data

⇒ in ML $\Rightarrow$ Decision Tree $\Rightarrow$ When we make decision Tree fully (To last node) it leads to overfitting $\Rightarrow$ So we perform post pruning & pre pruning to reduce overfitting.

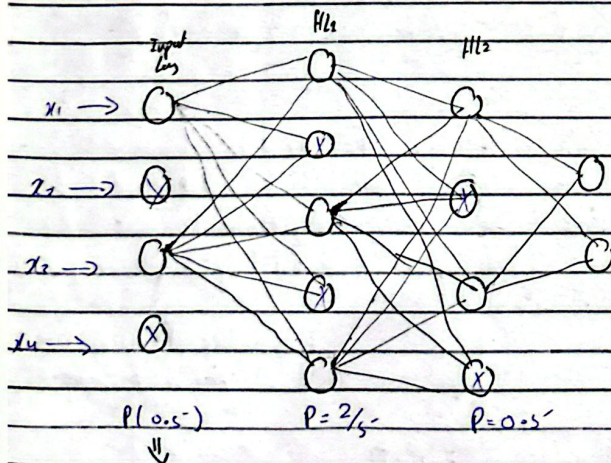
Note: See Decision Tree Notes & Random forest.

⇒ Random Forest: Many Decision Tree but we do Feature sampling & Row sampling. means give each decision Tree different sample of Data.

⇒ W.r.t ANN in Deep Neural Network ⇒ We have some many weights & bias ⇒ also some goes to overfitting.

→ How do we reduce it?
* We use drop out layer.

*** We De-activate Neurons in Different 'layer' by defining probability for deactivation of Neuron in each layer.



⇒ Probability of deactivation is 50% ⇒ means 50% of neurons will be deactivated e.g. if 4 then 2- deactivate.

H1

⇒ $P = \frac{2}{3}$ ⇒ Probability is of $\frac{2}{3}$ neurons will be deactivated

⇒ H2

$P = 0.5$ ⇒ 50% probability of neurons being deactivated.

⇒ In 2nd iteration ⇒ The probability is randomly initialized for each layer & the neurons are selected randomly

⇒ In 3rd iteration ⇒ again probability is random & so on.

⇒ Selecting Probability for drop-out is based on hyper parameter.

⇒ It is done to reduce overfitting.

⇒ This is done only for Train Data ⇒ reduce overfitting.

H₁

$\Rightarrow P = \frac{2}{5} \Rightarrow$ Probability is of $\frac{2}{5}$ neurons will be deactivated

$\Rightarrow$ H₂

P.O.S $\Rightarrow$ 50% probability of neurons being deactivated.

$\Rightarrow$ In 2nd iteration $\Rightarrow$ The probability is randomly initialized for each layer & the neurons are selected randomly

$\Rightarrow$ In 3rd iteration $\Rightarrow$ again probability & selection is random & so on.

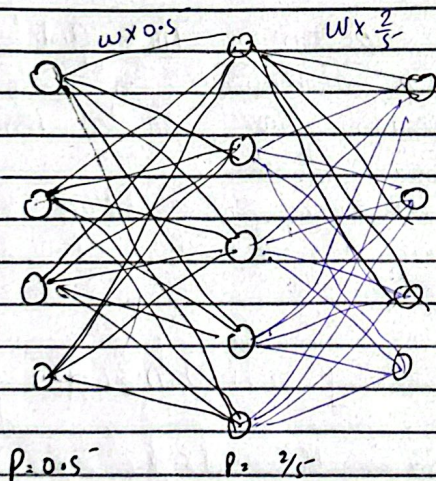
$\Rightarrow$ Selecting Probability for drop-out is based on hyper parameter.

$\Rightarrow$ It is done to reduce overfitting.

$\Rightarrow$ This is done only for ~~Test~~ Train Data $\rightarrow$ reduce overfitting.

⇒ For Test Data When Making predictions, we don't deactivate neurons.

instead we multiply the weights with probability



↳ for this layer

so for this layer

$W \times 0.5$

$W \times \frac{2}{5}$

⇒ Convolutional Neural Network (CNN)

1) ANN → Supervised learning → classification
→ Regression

2) CNN → Input is in form of images
e.g. Image classification
Object detection, segmentation

→ Cerebral Cortex & Visual

→ Visual Cortex: Region of Brain receives, integrates & processes visual info from retina

Visual Cortex has 5 layers
 $V1 - V5$